AI Devops Assistant

Deploy it using the ECS and ECR using jenkins

🏗️ ARCHITECTURE OVERVIEW

1. Launch EC2 (Amazon Linux 2, t2.medium, 30GB)
2. Install Java, Jenkins, Docker, Git
3. Attach IAM Role to EC2
4. Create ECR Repository
5. Create Jenkins Pipeline (Push Image to ECR)
6. Create ECS Cluster (Fargate)
7. Create Task Definition
8. ECS Service
9. Application Load Balancer
10. Public URL

✅ STEP 1: Launch EC2 Instance

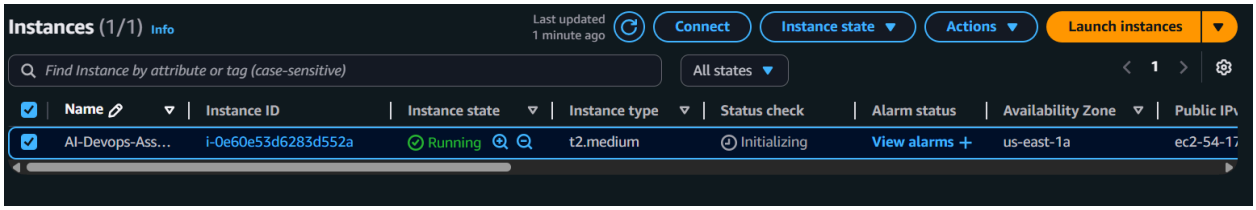
Go to:

👉 **AWS Console → EC2 → Launch Instance**

Configure:

- **Name**: Jenkins-Server
- **AMI**: Amazon Linux 2
- **Instance type**: t2.medium
- **Storage**: 30 GB
- **Key pair**: Select or create
- **Security Group**:
  - Allow:
    - SSH (22)
    - Custom TCP (8080) → Jenkins
    - Custom TCP (8000 or 8501) → App
    - HTTP (80) optional

      Click **Launch Instance**



✅ STEP 2: Create IAM Role for EC2

Go to:

👉 **IAM → Roles → Create Role**

Select:

- Trusted entity: **AWS Service**
- Use case: **EC2**

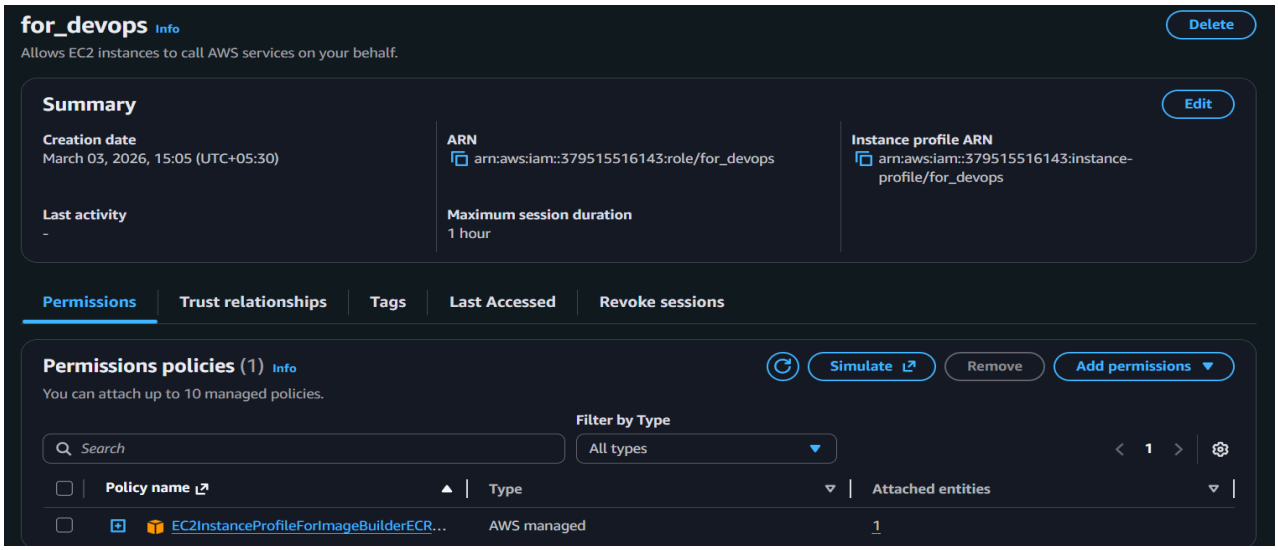
Click Next

Attach These Policies:

- `AmazonEC2ContainerRegistryFullAccess`
- `AmazonECS_FullAccess`
- `AmazonSSMManagedInstanceCore`

You can also create a custom policy if needed, but above works fine for DevOps setup.

Name it: jenkins-ecs-ecr-role

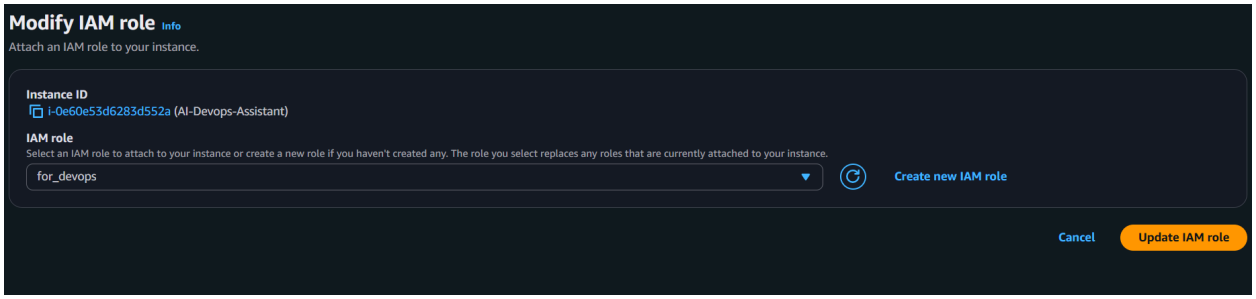


✅ STEP 3: Attach IAM Role to EC2

1. Go to **EC2**
2. Select your instance
3. Click **Actions** → **Security** → **Modify IAM Role**
4. Attach: `jenkins-ecs-ecr-role`
5. Save

⚠️ VERY IMPORTANT — without this, ECR push will fail.

Attach the role



✅ STEP 4: Connect to EC2
ssh -i your-key.pem ec2-user@<public-ip>

✅ STEP 5: Install Java 21

```
sudo yum update -y
sudo yum install java-21-amazon-corretto-headless -y
sudo yum install java-21-amazon-corretto-devel -y
java --version
```

✅ STEP 6: Install Jenkins
Follow official docs from Jenkins:

https://www.jenkins.io/doc/book/installing/linux/

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo

sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2023.key

sudo yum upgrade -y
```

sudo yum install jenkins -y

Start Jenkins:

sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins

✅ STEP 7: Install Docker

sudo yum install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo systemctl status docker

Add users to docker group:

sudo usermod -aG docker ec2-user
sudo usermod -aG docker jenkins

sudo yum install git -y

Restart Jenkins:

sudo systemctl restart jenkins

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Install suggested plugins

Create admin user

✅ STEP 9: Install Required Jenkins Plugins

Go to:

Manage Jenkins → Plugins → Install:

- Docker Pipeline
- Git
- Pipeline
- AWS Credentials
- Amazon ECR

Restart Jenkins.

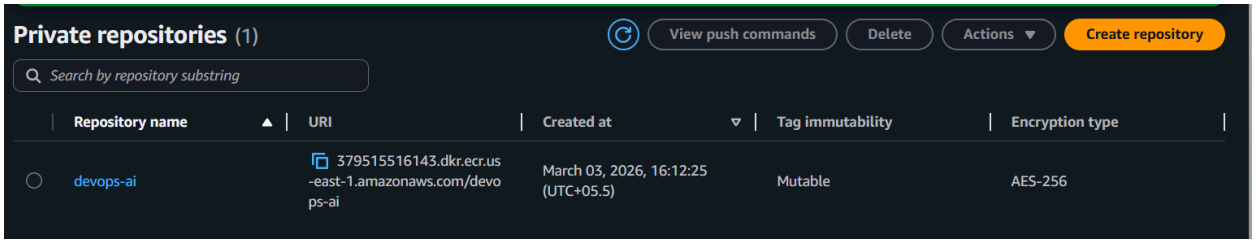
✅ STEP 10: Create ECR Repository

Go to:

👉 Amazon Elastic Container Registry

Click **Create Repository**

Name:



Devops-ai

379515516143.dkr.ecr.us-east-1.amazonaws.com/devops-ai

✅ STEP 11: Dockerfile

```dockerfile
FROM python:3.10-slim AS build

WORKDIR /app

RUN rm -rf /var/lib/apt/lists/*
COPY requirements.txt .

RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.10-slim

WORKDIR /app

COPY --from=build /install /usr/local/

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY main.py .

EXPOSE 8501

CMD ["streamlit", "run", "main.py"]
```

✅ STEP 12: Jenkins Pipeline (Push to ECR)

Create → New Item → Pipeline

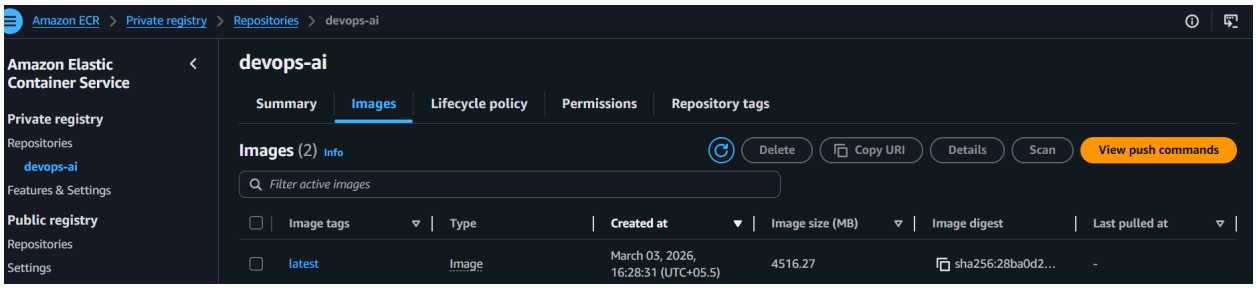
Paste this:

```
pipeline {
  agent any

  environment {
    AWS_REGION = "us-east-1"
    ECR_REPO = "379515516143.dkr.ecr.us-east-1.amazonaws.com/devops-ai"
    IMAGE_TAG = "latest"
  }

  stages {
```

```
    stage('Checkout Code') {
        steps {
            git branch: 'main',
                url: 'https://github.com/mantasha0cloud/AI-devops_Assistant.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh '''
                docker build -t allenaira/devops_assistant:new .
            '''
        }
    }

    stage('Login to ECR') {
        steps {
            sh '''
                aws ecr get-login-password --region $AWS_REGION | \
                docker login --username AWS --password-stdin $ECR_REPO
            '''
        }
    }

    stage('Tag Image') {
        steps {
            sh '''
                docker tag allenaira/devops_assistant:new $ECR_REPO:$IMAGE_TAG
            '''
        }
    }

    stage('Push to ECR') {
        steps {
            sh '''
                docker push $ECR_REPO:$IMAGE_TAG
            '''
        }
    }
}

post {
    success {
```

```
            echo "Image successfully pushed to ECR!"
        }
        failure {
            echo "Pipeline Failed!"
        }
    }
}
```

Images is push to ECR



✅ STEP 13: Create ECS Cluster (Fargate)

Go to:

👉 Amazon Elastic Container Service

1. Click Clusters
2. Create Cluster
3. Choose **Networking only (Fargate)**
4. Name: devops-cluster
5. Create

✅ STEP 14: Create Task Definition

1. ECS → Task Definitions
2. Create new → Fargate

Configure:

- Task name: devops-task
- CPU: 0.5 vCPU
- Memory: 1GB
- Execution Role: Create new (auto)

Container Config:

- Name: app
- Image: 379515516143.dkr.ecr.us-east-1.amazonaws.com/devops-ai:latest

Port mapping:

- Container port: 8501

Health Check:

CMD-SHELL, curl -f http://localhost:8501 || exit 1

Create Task.

Create ECS Service with Load Balancer

Go to:

👉 Amazon Elastic Container Service
→ Clusters → Select your cluster

Click **Create Service**

🔹 1. Configure Service

- Launch type: **Fargate**
- Task Definition: Select your created task
- Service name:
- devops-ai-service
- Desired tasks: 1

Click Next

🔹 2. Networking Configuration

- Cluster VPC: Select default VPC (or your custom)
- Subnets: Select **at least 2 public subnets**
- Security Group:
    - Create new OR use existing
    - Allow:

        - HTTP (80) from 0.0.0.0/0

- Auto-assign Public IP: **DISABLED**
  (Because Load Balancer will handle traffic)

Click Next

🔹 3. Load Balancer Configuration

Select:

✅ **Application Load Balancer**

Choose:

- Create new load balancer
- Name:

devops-ai-alb

Listener port: **HTTP 80**

Target group:

- Create new
- Target type: IP
- Port: **8501**

🔄 What Happens Now?

ECS will:

1. Create ALB
2. Create Target Group
3. Register your container IP
4. Perform Health Checks
5. Route traffic to container

✅ STEP 16: Access Application via Load Balancer

Go to:

👉 Elastic Load Balancing
→ Load Balancers

Copy:

DNS name

Open in browser:

http://<load-balancer-dns>

ALB Security Group:

Allow:

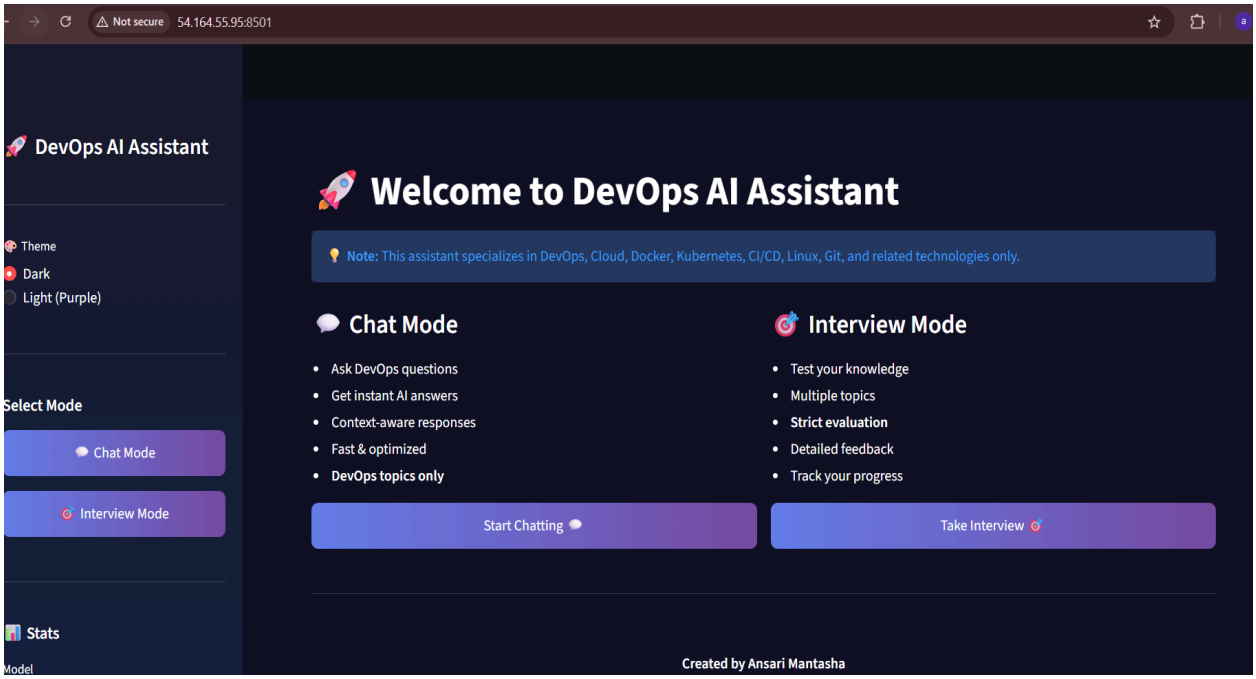
- HTTP (80) from 0.0.0.0/0

ECS Task Security Group:

Allow:

- 8501 from ALB Security Group (NOT from 0.0.0.0/0).

It takes **2–5 minutes**



And You are Done!!!